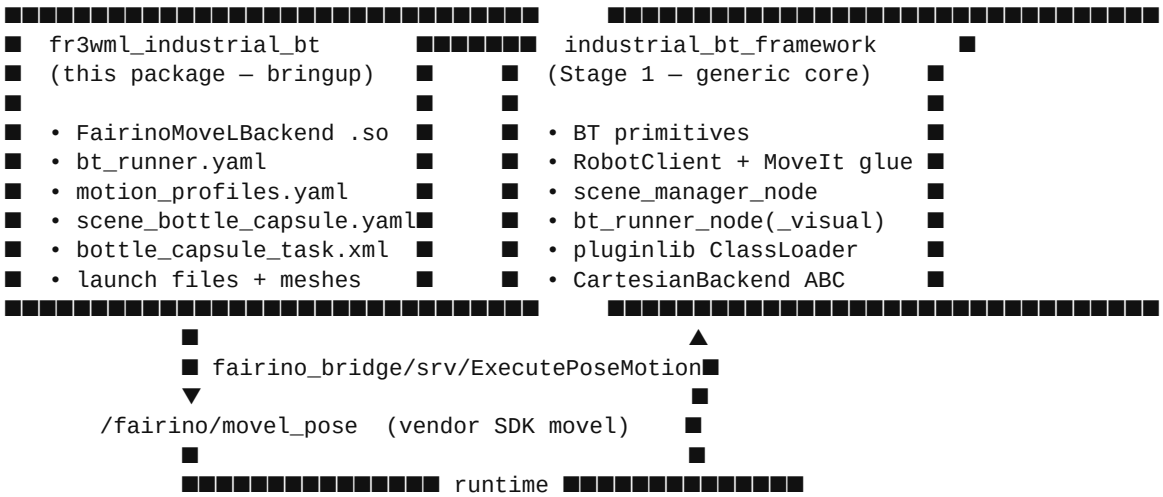
Stage 2 Guide — `fr3wml_industrial_bt`

> *How the generic `industrial_bt_framework` is adapted to drive the real Fairino FR3WML robot for the bottle + capsule pick-and-place, replicating every step of the monolithic `fr3wml_camera_gripper` application — without editing framework code.*

---

1. Architecture

Two packages, clear boundaries.



Runtime topology (FR3WML demo):

```
(user terminals)
  ros2 run ros2_cmd_server ros2_cmd_server
  ros2 run fairino_bridge  fairino_bridge

(single launch)
  ros2 launch fr3wml_industrial_bt industrial_bt_demo.launch.py
      → move_group + RSP + RViz + ros2_control controllers   (t=0)
      → scene_manager_node (framework)                       (t≈5 s)
      → bt_runner_node     (framework, loads FairinoMoveLBackend)
                                                             (t≈10 s)
```

---

2. What the demo does

The BT ticks through the same sequence as the monolith:

1. PTP to `pre_pick_bottle_joints_deg` (MoveIt) 2. Remove scene `bottle` (so the descent has no collision) 3. Linear descent (Fairino `movel`, 80 %) to the bottle grasp TCP 4. Activate gripper (Trigger `/gripper/close`) 5. Linear retreat 0.4 m in tool frame (60 %) 6. PTP to `pre_place_bottle_joints_deg` 7. Linear down (50 %) to the place TCP 8. Release gripper (Trigger `/gripper/idle`) 9. Teleport scene `bottle` to its place pose 10. Linear retreat 0.1 m (80 %) 11. PTP to `pre_pick_capsule_joints_deg` 12.

Remove scene `capsule` 13. Linear descent (80 %) to capsule grasp TCP 14. Activate suction (Trigger `/suction/on`) 15. Linear retreat 0.05 m (80 %) 16. PTP to `pre_place_bottle_joints_deg` (reused) 17. Linear down (40 %) to capsule place TCP 18. Release suction (Trigger `/suction/off`) 19. Teleport scene `capsule` to its place pose 20. Linear retreat 0.05 m (80 %) 21. MoveIt named target `pos1` (home)

All numbers are those of the monolith; none of them live in C++.

---

3. Folder layout

```
fr3wml_industrial_bt/
■■■ CMakeLists.txt
■■■ package.xml
■■■ plugins/
■   ■■■ fairino_movel_backend.cpp            ← the ONLY new C++
■   ■■■ fr3wml_industrial_bt_plugins.xml
■■■ launch/
■   ■■■ industrial_bt_demo.launch.py         ← headless
■   ■■■ industrial_bt_visual_demo.launch.py  ← + Groot2
■■■ bt_trees/
■   ■■■ bottle_capsule_task.xml
■   ■■■ bottle_capsule_task_visual.xml
■■■ config/
■   ■■■ bt_runner.yaml
■   ■■■ motion_profiles.yaml
■   ■■■ scene_bottle_capsule.yaml
■   ■■■ fairino_movel_backend.yaml           ← plugin YAML
■■■ meshes/bottle/…   crate/…   gripper2.stl, suction_cap.stl, kinetic.*
■■■ urdf/fr_camera_gripper.urdf.xacro        ← copied for reference
■■■ rviz/config_fr_camera_gripper.rviz       ← copied for reference
```

---

4. Framework-adaptation summary

Every degree of freedom the monolith exposed, mapped to the corresponding framework extension point. This is the list of *all* adaptations — nothing is buried in C++.

Dimension	Adaptation	Extension point
Vendor LIN motion (Fairino SDK `movel`)	New `FairinoMoveLBackend` plugin calling `fairino_bridge::srv::ExecutePoseMotion` on `/fairino/movel_pose` with `motion_type="LINEAR"`	`CartesianBackend` ABC + pluginlib
Wrist3 → flange offset (0.098 m)	`flange_offset_xyz: [0.0, 0.0, 0.098]` in `fairino_movel_backend.yaml`	Plugin YAML params
Planning group / home named target	`planning_group: fr3wml`, `home_named_target: pos1` in `bt_runner.yaml`	Framework params
Gripper / suction Trigger services	`tools.gripper_primary.*` and `tools.suction_primary.*` in `bt_runner.yaml`	`ToolRegistry`

Dimension	Adaptation	Extension point
Joint targets, pick/place XYZs, retreats, ids	`task_parameters.*` in `bt_runner.yaml` → pushed to BT blackboard under short keys	`loadTaskParametersToBlackboard`
Per-segment `speed_percent` (80/60/50/80/…)	Distinct profiles (`pick_bottle`, `retreat_bottle_pick`, …) in `motion_profiles.yaml`	`MotionProfileRegistry`
TCP tool offsets (softgripper / suctioncup)	`softgripper_tcp_offset_xyz: [0,0,0.19]`, `suctioncup_tcp_offset_xyz: [0,0,0.19]`; bound to `<ComputeTCPTarget>`	`ComputeTCPTarget` primitive
Scene (table, walls, crate, bottle, capsule)	`scene_bottle_capsule.yaml` with mesh/box/cylinder entries	`scene_manager_node` schema
ACM links (gripper + suction)	`attach_acm_links: [softgripper_link, suctioncup_link, gripper_body, suction_cap]`	`scene_manager_node`
Dynamic object removal / teleport	`dynamic: true` + `<RemoveSceneObject>` / `<TeleportSceneObject>` in BT XML	Scene primitives + scene_manager
Scene settle delay	`scene_update_wait_ms: 800` in both files	Params
Task sequence	`bottle_capsule_task.xml` — PickPart/PlacePart SubTrees reused 2×	BT XML only

Framework code edits: zero.

---

5. The one new C++ file — `FairinoMoveLBackend`

File: `plugins/fairino_movel_backend.cpp`.

Subclass of `industrial_bt_framework::CartesianBackend`. It exposes one service client (`rclcpp::Client<fairino_bridge::srv::ExecutePoseMotion>`) and one override:

```
bool executeLinear(
    const geometry_msgs::msg::Pose & tcp_target_pose,   // TCP in base_link
    const industrial_bt_framework::MotionProfile & p,   // movel_speed_percent
    const std::string & desc) override
{
  auto flange = tcpToFlange(tcp_target_pose);        // apply wrist3→flange offset
  auto req = std::make_shared<ExecutePoseMotion::Request>();
  req->target_pose               = flange;
  req->motion_type               = "LINEAR";         // SDK movel
  req->speed_percent             = p.movel_speed_percent;
  req->tool_id                   = tool_id_;
  req->user_id                   = user_id_;
  req->load_frames_before_motion = load_frames_before_motion_;
  auto resp = client_->async_send_request(req).get(/* timeout */);
  return resp->success;
}
```

The `tcpToFlange` helper rotates `flange_offset_xyz` by the TCP orientation (tf2 quaternion) and subtracts, so the framework stays TCP-centric while the vendor service sees flange poses.

Registration:

```
<!-- plugins/fr3wml_industrial_bt_plugins.xml -->
<library path="fr3wml_industrial_bt_backends">
  <class name="fr3wml_industrial_bt/FairinoMoveLBackend"
         type="fr3wml_industrial_bt::FairinoMoveLBackend"
         base_class_type="industrial_bt_framework::CartesianBackend"/>
</library>
```

`CMakeLists.txt`:

```
pluginlib_export_plugin_description_file(
  industrial_bt_framework plugins/fr3wml_industrial_bt_plugins.xml)
```

The framework's `bt_runner_node` loads it at startup via `pluginlib::ClassLoader<CartesianBackend>` — selection is purely YAML-driven (`cartesian_backend: "fr3wml_industrial_bt/FairinoMoveLBackend"`).

---

6. The BT tree

File: `bt_trees/bottle_capsule_task.xml`.

It uses only framework primitives (`MoveToJointTarget`, `ExecuteCartesianSegment`, `ComputeTCPTarget`, `OffsetPoseInToolFrame`, `ActivateTool`, `ReleaseTool`, `RemoveSceneObject`, `TeleportSceneObject`, `MoveToNamedTarget`, `LogMessage`).

Two small SubTrees do all the work:

- `PickPart(pre_pick_joints, part_id, pick_xyz, tool_offset, tool, retreat_m, pick_profile, retreat_profile)`
- `PlacePart(pre_place_joints, part_id, place_xyz, tool_offset, tool, retreat_m, place_profile, retreat_profile)`

`MainTree` just instantiates them 2 × (bottle → capsule) with different remappings. Adding another part to the task ≡ adding one more `<SubTree>` call and a handful of `task_parameters:` entries.

---

7. Groot2 workspace folder

Lives at `/home/fra/fr3wml_ws/groot2/` (outside any ROS 2 package):

```
groot2/
■■■ fr3wml_industrial_bt.btproj
■■■ README.md
■■■ trees/  (symlinks to src/fr3wml_industrial_bt/bt_trees/*.xml)
```

```
■■■ models/industrial_bt_framework_nodes.xml
■■■ logs/   (drop /tmp/bt_trace.btlog here to replay)
```

Workflow:

- **Live**: launch the visual demo, then Groot2 → Monitor → `tcp://localhost:1666`.
- **Replay**: `cp /tmp/bt_trace.btlog groot2/logs/<date>.btlog`, then Groot2 → Log Viewer.
- **Edit**: open `.btproj`, edit the visual tree (it edits the real XML through the symlink), relaunch — `--symlink-install` already picks it up.

---

8. Build & run

```
cd /home/fra/fr3wml_ws
colcon build --packages-select fr3wml_industrial_bt --symlink-install
source install/setup.bash
```

Prerequisites (user terminals):

```
ros2 run ros2_cmd_server ros2_cmd_server
ros2 run fairino_bridge fairino_bridge
```

Headless run:

```
ros2 launch fr3wml_industrial_bt industrial_bt_demo.launch.py
```

Visual run (Groot2):

```
ros2 launch fr3wml_industrial_bt industrial_bt_visual_demo.launch.py
# Groot2 → Monitor → tcp://localhost:1666
```

Success criterion: the BT ends with `■■ BT finished: SUCCESS ■■` and the final robot pose is `pos1`, while the scene objects (bottle + capsule) stay in their final placed poses (force-republish at 1 Hz prevents them from disappearing).

---

9. Extending

- **Different parts / poses**: add entries to `task_parameters:` and one extra `<SubTree>` instantiation in the BT — no C++ change.
- **Different tool**: add a new `tools.<name>` entry (Trigger services) and bind `tool="<name>"` in the BT XML.
- **Different robot**: swap `moveit_config_pkg`, `planning_group`, `home_named_target`; if the vendor has no MoveIt Cartesian support, write a new `CartesianBackend` plugin (1 file; `FairinoMoveLBackend` is the template).
- **Different scene objects**: add a new block in `scene_bottle_capsule.yaml` (`box`, `cylinder`, `sphere`, or `mesh`) — the generic `scene_manager_node` handles it with no code change.

The monolith's behaviour is now one C++ file and four YAML files away from any other industrial pick-and-place.